MASTER ACTION PLAN - EXECUTION PLAYBOOK

IC Master Action Plan

Every important finding from the research and the deep audit, converted into one ordered build order: five phases from honest tests to go-live, the rhythm that runs the bots, and the trust curve toward autonomy.

Standard-autonomous Repository - What To Do, In Order

Phases 0-5 - Operating Rhythm - Vertical Expansion - Library Map

OCTOBER 2026

Table of Contents

1. The One-Page Plan 1

2. Phase 0 - Where You Stand Today 2

3. Phase 1 - Week 1: Truth First 4

4. Phase 2 - Week 2: Walls and Hard CI Rules 6

5. Phase 3 - Week 3: One House, One Pipeline 8

6. Phase 4 - Week 4: Face and Muscle (Go-Live) 10

7. Phase 5 - Day 30+: The Agent Trust Curve 12

8. The Operating Rhythm 13

9. The Vertical Expansion Recipe 14

10. The Library Map - Which File Matters When 16

11. The Master Checklist 17

1. The One-Page Plan

This is the plan of record. The deep audit told you what is broken and why; the Agent Engineering Playbook told you the theory behind the method; this document tells you what to do, in order, without needing either one open. It converts every important finding into five phases of concrete work: truth first (week 1), walls and CI (week 2), one house and one pipeline (week 3), go-live (week 4), and the trust curve that graduates your bots toward autonomy (day 30 onward). Each phase ends in a shippable state, and every step carries its own acceptance check so you always know whether you are done. If you read only one page before starting work, read the Master Checklist in Chapter 11 and work it top to bottom.

The operating method is the Lauren Tan loop, and every phase obeys it: verification before trust, trust before autonomy, and hard CI rules before anything merges or sends without you. In practice that means week 1 makes the repository honest, week 2 builds the security walls and the CI gate that blocks bad work, week 3 collapses three module trees into one and wires the eight-stage pipeline, week 4 turns the product on for fifty real merchants, and only then does autonomy begin to climb the five-stage trust curve. Skipping ahead is the one forbidden move, because the CI gate built in week 2 is the load-bearing dependency for every later week: it is what allows a two-to-five person team to ship agent-made changes without fear. The weekly ritual that keeps the whole machine honest is described in Chapter 8, and it takes thirty minutes.

5	28	50	5
phases, one build order	days to go-live	merchant pilot	trust-curve stages

Deliverable	What it holds	Open it when
IC Master Action Plan (this document)	The ordered build order: steps, commands, acceptance checks, KPIs	Every work session; it is the plan you execute
IC Intelligence Center Audit and Upgrade Report	The evidence: 9 chapters with defects D1-D8, the 12-dimension scorecard, UI laws, verticals	When you need the why behind a step or full defect detail
Agent Engineering Playbook: Research and Build Blueprint	The theory: the Lauren Tan masterclass, the Grok marketplace, skills, verification	When designing a new bot or skill, or onboarding a teammate

Table 1: The three deliverables and the job of each - do, why, theory.

Use the document mechanically. Work in order, do not parallelize across phases, and treat each acceptance check as a gate: when the check passes, move on; when it fails, the step is not done no matter how much code exists. Every step names its files, its commands and the evidence that proves completion,

in the same spirit the repository itself uses: a claim without proof is not a finished step. Where a step touches a file that the audit flagged, the defect number (D1 to D8) is given so you can cross-reference the audit report for full detail.

2. Phase 0 - Where You Stand Today

Before any work, fix the starting line. The audit measured IC across twelve dimensions and the shape of the result is unusual: the agent architecture scores at the top of the scale while packaging, security and test integrity score near the bottom. That combination is good news, because the architecture is the expensive part to rebuild and the weak dimensions are a bounded, mechanical backlog. It also explains the strategy of this plan: two weeks of plumbing before a single new feature, then the product goes live on a foundation a big company would accept.

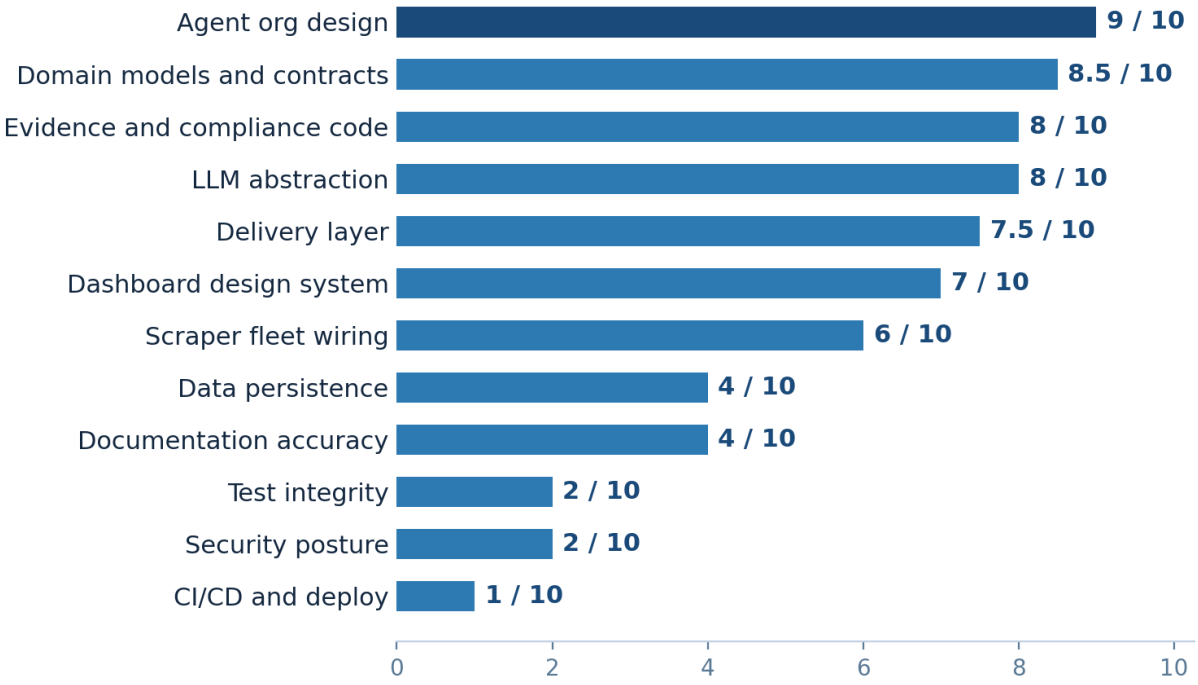


Figure 1: The 12-dimension audit scorecard from the deep audit (0-10 per dimension).

What is genuinely built is rare. Most agent repositories are a demo script with a for-loop; IC already runs a real organization of machines:

- A five-agent organization - Scout, Auditor, Copywriter, ApprovalGate, Closer-Hermes - each with a single mandate, on the Grok marketplace pattern.
- Typed contracts: a HandoffEnvelope carries every handoff, so agents cannot mangle each other's data.

- The pstack evidence chain: every audit finding becomes a VerifiedClaim with a proof reference and a PROVEN_TRUE status, assembled into a PstackAuditBundle with a proof score.
- The ECC shield on both sides of the wall: AgentShield scans code before merge; the runtime gate checks every outbound message for grounded claims and hygiene.
- Fifty-one vendored skills in .agents/skills/ - pstack, superpowers, mattpocock-skills, ecc, treg, harness2 - the training manuals of the team.
- A 6,528-line dashboard with light/dark design tokens, SpotlightCard and the Paperclip Live Studio drawer bridging the agent runtime.

Defect	What it is	Severity
D1	Test integrity: README claims 51 of 51 passing; measured 33 passed, 1 failed, 8 errored, 5 files uncollectable	Critical
D2	Missing module: src/storage/repository.py (LeadRepository, SessionRepository) never committed; root cause of the 5 dead test files	Critical
D3	Three module trees: v1 at the root (core/, services/, scrapers/, delivery/), v2 in src/, orphan outreach/ nobody imports	High
D4	Zero security: no auth on FastAPI, CORS wildcard, uvicorn reload=True, /docs exposed, PII in plaintext JSON	Critical
D5	Repo hygiene: 89 data/logs files committed (9.8 MB of lead dumps plus 572 KB of logs)	High
D6	Silent degradation: scrapling and firecrawl missing from requirements; scrapers fall back without a trace	High
D7	No CI, no Dockerfile, no LICENSE; README links point to file:///c:/Users/... paths	Medium
D8	Dashboard monolith: one 6,528-line file holds the entire UI, blocking review and redesign	Medium

Table 2: The eight defects from the audit; every one is scheduled in a phase of this plan.

The rule for Phase 0 is one sentence: do not build features on dishonest tests. Every new capability added before week 1 lands becomes part of the debt, because you will be unable to prove it works and unable to refactor around it safely. The thirty-day plan spends its first two weeks entirely on this backlog for exactly that reason, and the trust curve in Chapter 7 depends on the honesty established here - a bot can only earn autonomy on top of measurements you believe.

3. Phase 1 - Week 1: Truth First

The week has one goal: make the repository tell the truth. Today the README promises 51 passing tests; the measurement is 33 passed, 1 failed, 8 errored, and 5 test files that cannot even import (defect D1). The root cause is a single missing module (D2), which makes this the highest-leverage week in the plan: one file, written properly, repairs the test suite, unblocks honest delivery code, and gives the storage layer the repository pattern it was designed for. The steps are ordered so each one makes the next easier, and none of them requires new features.

Step 1.1 Write the missing repository module

Create src/storage/repository.py with exactly two classes. LeadRepository owns leads: add, get, update_status, list_by_vertical. SessionRepository owns audit sessions: open, append_event, close. Every method is typed and every mutation returns the updated record, so callers never hold stale state. This module is the IC's memory - leads must survive restarts, and the audit trail must be queryable - which is why five test files were written against it before it existed. A JSON-backed implementation is enough for week 1; upgrade the storage engine later without touching callers, because they only see the interface.

```
# src/storage/repository.py - the module five test files are waiting for
from typing import Optional
from src.models import Lead, Session, LeadStatus
class LeadRepository:
    def add(self, lead: Lead) -> Lead: ...
    def get(self, lead_id: str) -> Optional[Lead]: ...
    def update_status(self, lead_id: str, status: LeadStatus) -> Lead: ...
    def list_by_vertical(self, vertical: str) -> list[Lead]: ...
class SessionRepository:
    def open(self, session: Session) -> Session: ...
    def append_event(self, session_id: str, event: dict) -> Session: ...
    def close(self, session_id: str) -> Session: ...
```

Step 1.2 Make pytest honestly green

Run the suite and read the five collection errors; they should disappear with Step 1.1. Fix the one failing delivery test and the eight errors on their own merits - do not skip or delete tests to get green, because the suite is the scoreboard the trust curve will play on. Then correct the README: replace the 51-passed claim with the real numbers and the command that reproduces them. The README rule from now on: no number appears in the README that the command line cannot reproduce.

```
python -m pytest tests/ -q              # expect: every file collects, counts p
rinted
python -m pytest tests/ -q --tb=short   # fix failures by reading, not skipping
```

Step 1.3 Clean the working tree

The 89 committed data and logs files (D5) are history pollution and a privacy risk: lead dumps with phone numbers belong in storage, not in git. Untrack them, ignore them, and commit. Be honest about the limit of this step: the files remain in the git history, so if the repository will ever be public, schedule a filter-repo pass as week-4 cleanup. The immediate win is that a fresh clone no longer ships customer data.

```
git rm -r --cached data logs
printf 'data/\nlogs/\n*.log\n.env\n' >> .gitignore
git commit -m "stop tracking data and logs"
```

Step 1.4 Pin the scraper dependencies

scrapling and firecrawl are imported but never declared (D6), so a fresh install silently degrades the whole scraper fleet to fallback sources and nothing in the logs explains why the lead quality dropped. Declare both with pinned versions in requirements.txt, and add a startup health check that fails loudly when a declared engine cannot import. After this step, a missing dependency is an error at boot, not a mystery at the end of the week.

- pytest exits 0 with every file collected, and the README prints the same counts the command produces.
- git status shows no data or logs after a full pipeline run.
- A fresh virtualenv installs from requirements.txt and every scraper engine imports cleanly.
- grep -r "51 passed" README.md returns nothing.

> *Truth first: a repository that lies to itself cannot be trusted with customers.*

4. Phase 2 - Week 2: Walls and Hard CI Rules

In the masterclass, the path to letting agents merge while you sleep is verification, then trust, then hard CI rules. IC already has verification (the pstack chain) and a human gate, but it has no walls and no gate that a machine cannot walk around: the API is unauthenticated (D4) and no automated check stands between a commit and the main branch (D7). This week builds both, and it is the week that makes every later promise legal - stage 3 of the trust curve can only auto-send because the ECC scan guards the pipeline, and stage 4 can only auto-merge because the CI gate is green.

Step 2.1 Lock the API

The audit found lead data with phone numbers readable by anyone who finds the port, so the lock comes first. Add a dependency to the FastAPI app that checks an API key on every route, reject requests without it with 401, and load the key from .env. Restrict CORS to the dashboard origin and delete the wildcard. Run uvicorn without reload outside development, and either disable /docs in production or put it behind the key. None of this slows the agents down; it slows down everyone you did not invite.

- Add an API-key dependency to every route; missing or wrong key returns 401.
- CORS allowlist contains only the dashboard origin; the wildcard is deleted.
- uvicorn runs without reload in any deployment; /docs is off or behind the key.
- The key lives in .env and is never hardcoded - the ECC scan in Step 2.3 enforces this forever.

Step 2.2 Containerize

Write a Dockerfile from python:3.12-slim, install from the pinned requirements, create a non-root user, and declare one entrypoint. Add a docker-compose file with three services - api, worker, dashboard - so the whole product comes up with one command from a clean clone. Containers are not glamour work; they are what makes the onboarding command in the README true and what makes the CI runner and any future VPS behave identically to your laptop.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -m ic && USER ic
CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 2.3 The CI gate

Create a GitHub Actions workflow that runs on every push and pull request: the full pytest suite, ruff, and the ECC AgentShield scan for hardcoded credentials, injection-prone calls and undeclared imports - the scan that would have caught D6 before it shipped. Then enable branch protection so a red build blocks the merge. This is the masterclass's "hard CI rules" implemented verbatim, and it is the precondition for stage 4 of the trust curve, where cleanup PRs merge themselves only when this gate is green.

```
name: ci
on: [push, pull_request]
jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - run: pip install -r requirements.txt && pip install pytest ruff
      - run: python -m pytest tests/ -q
      - run: ruff check src/
      - run: python -m ecc_shield.scan src/
```

Step 2.4 Hygiene: LICENSE, links, protection

Close the loose ends that make the repository look abandoned. Add an MIT LICENSE so nobody has to ask. Replace every file:///c:/Users/... link in the README with in-repo paths, because those links are dead on every machine except the one that wrote them. Enable branch protection requiring the CI check from Step 2.3, and tag the state v0.1.0 - the first release whose claims you can defend.

- An unauthenticated request to any API route returns 401.
- docker compose up brings api, worker and dashboard up from a clean clone.
- A pull request with a failing test cannot merge, and the AgentShield scan runs on every PR.
- No file:/// links remain, the LICENSE exists, and v0.1.0 is tagged.

5. Phase 3 - Week 3: One House, One Pipeline

Three module trees cost double maintenance and make every change a negotiation between generations (D3). The merge follows the strangler-fig pattern: freeze the v1 tree at the root, port whatever main.py still needs, point the imports at src/, fold the orphan outreach/ scrapers into the fleet, and delete the old trees in one commit. The finished state has one direction of dependency - apps import packages, never the reverse - which is the property that makes big-company refactors cheap and, in week 4, makes the trust curve auditable.

Today	New home	Action
core/, services/, delivery/ (v1 root)	deleted	Port what main.py needs, re-point it at src/, then delete
scrapers/ (v1 root)	packages/scrapers/	Merge engines into the one scraper SDK, then delete
outreach/ (orphan tree)	packages/scrapers/	Keep its three unique scrapers, merge them, delete the tree
src/agents, src/contracts	apps/ic-worker/, packages/contracts	Promote to the worker app; contracts become a shared package
dashboard (6,528-line file)	apps/ic-dashboard/	Split into modules (Step 3.1) and move

Table 3: The migration map from three module trees to one house.

Step 3.1 Split the dashboard monolith

The 6,528-line file becomes about fifteen modules: the design tokens, SpotlightCard, ClickSpark, GlareHover, LeadTable, LeadDetail, ApprovalQueue, EvidenceChip, PaperclipDrawer (the Live Studio bridge), StudioConsole, AuditReportView, OutboxView, VerticalSwitcher, the API client, and the app shell. The rule is the same one-directional graph as the backend: the shell imports views, views import tokens, and nothing imports the shell. Beyond tidiness this unblocks the week-4 UI upgrades, because you cannot review a component diff with the visual-parity skill, or ship an anti-slop checklist per component, when every pull request touches the whole application.

Do the split mechanically and resist the urge to redesign while moving. The contract with the audit report's UI chapter is that the old screen and the new screen must be pixel-equivalent first, improved second: run the old dashboard, screenshot it, split the file, screenshot the result, and only then apply upgrades one component at a time. This is exactly the workflow the vendored pstack visual-parity skill exists for, and it turns a risky rewrite into a sequence of provable steps.

Step 3.2 One scraper SDK, four engines

Write one interface - scrape(source) returning one standard LeadSource schema - and put the four engines behind it: google-maps-scraper for discovery, Scrapling for the audit fetch, firecrawl for content extraction, and treg for recurring crawl orchestration. Every engine sits behind a feature flag with a health check, and a dead engine turns that check red instead of quietly rerouting to fallback (which was D6's second half). Downstream code never learns which engine ran, which means a better engine can be swapped in per vertical without touching the agents that consume its output.

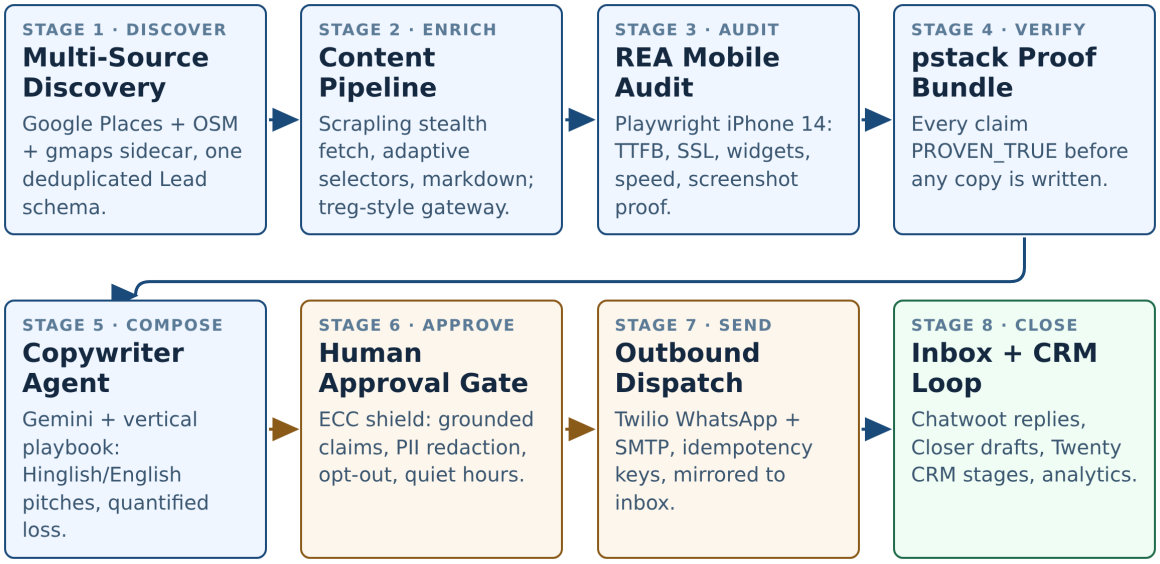


Figure 2: The eight-stage pipeline from discovery to CRM - the shape week 3 wires and week 4 fills.

Step 3.3 Make the proof bundle mandatory

Wire evidence_verifier at the copywriter-to-outbox edge: no message enters the Outbox without its PstackAuditBundle, and the Outbox UI shows the proof score beside every draft. This is the single change that makes the trust curve legal to climb, because autonomy in stage 3 is defined as sending only bundles whose evidence passed. It is also the cheapest possible defense against the one mistake this product cannot survive - a pitch line about a merchant's business that is simply false.

- One import tree: pytest is green from the new layout and no file imports upward.
- The dashboard builds from about fifteen modules; visual-parity screenshots match the old UI.
- One SDK interface with four engines behind flags; killing an engine turns its health check red.
- A draft without a PstackAuditBundle cannot enter the Outbox, and the proof score is visible in the UI.

6. Phase 4 - Week 4: Face and Muscle (Go-Live)

Week 4 is when the business turns on. The dashboard gets the five upgrades that make it feel like an agency operating system rather than a lab tool, delivery goes out through WhatsApp the legal way, replies land in a real inbox with a real CRM behind it, and a fifty-merchant pilot proves the whole eight-stage pipeline with humans still clicking send. Everything in this week assumes the three weeks before it: honest tests make the UI diffs reviewable, the CI gate makes the go-live changes safe, and the mandatory proof bundle makes every message defensible.

Step 4.1 The five UI upgrades that matter

Apply the top five upgrades from the audit's UI chapter, in this order, one component per pull request with a visual-parity screenshot attached:

1. Command palette (Cmd-K) for navigation - jump to any lead, approval or view; the pattern is standard in the navbar.gallery and openui references.
2. Lead table becomes a real data grid with filters and sticky header, on the watermelon ui pattern.
3. Approval cards grow evidence chips: the PROVEN_TRUE badge and proof score render beside each claim, from the hallmark and builtformars pattern.
4. Empty and error states are written like a human - "No leads yet. Run Discovery on clinics in Pune." - using the microcopy reference for tone.
5. ClickSpark fires on Approve and Send - one signature motion per screen, per the anti-slop motion budget.

Alongside the upgrades, the anti-slop laws become a pull-request template: contrast at or above 4.5 to 1, spacing tokens only, one motion signature per screen, dark and light parity, real data in every mockup. Ten checklist lines, fifteen seconds per review, and the dashboard cannot quietly drift into generic AI slop. The full ten laws and their sources are in Chapter 6 of the audit report.

Step 4.2 WhatsApp delivery, the legal way

Send through the official WhatsApp Business Cloud API, not unofficial automation - the account you are protecting is the business itself. Use pre-approved template messages, keep one message per business per week, and put a plain opt-out line in every note. Personalization comes from evidence, not flattery: "your booking link has been a 404 since March" is a sentence the PstackAuditBundle can prove, which is exactly why the bundle was made mandatory in Step 3.3. The ApprovalGate stays in the path this week; stage 3 of the trust curve is what may eventually automate it.

Step 4.3 Inbox and CRM

Wire Chatwoot as the inbox - self-hosted, with the WhatsApp channel built in - and connect its webhook so every reply flows back into IC and appears in the dashboard. Wire Twenty as the CRM: leads enter through its API with the pain points and proof scores attached, and the pipeline stages mirror the bot organization (discovered, audited, contacted, replied, won, lost). The rule that makes the pilot measurable: nothing lives only in a DM. If a conversation is not in Chatwoot and a deal is not in Twenty, it did not happen.

Step 4.4 The fifty-merchant pilot

Run the pilot on two verticals with sharp, detectable pain: clinics and coaching institutes, twenty-five merchants each, discovered with google-maps-scraper, audited with the REA mobile inspector, written by the CopywriterAgent, approved by you, sent through WhatsApp. Measure three numbers daily: reply rate, evidence pass rate, and human minutes per approved send. The reply-rate target for the pilot is ten percent or better; the human-minutes target is five or less, down from roughly fifteen when the audit began.

Metric	Week 0 (measured)	Week 4 (target)
Test suite	33 passed, 1 failed, 8 errored, 5 files dead	All files collect, all tests pass, counts printed in README
Security	0 protected routes, CORS wildcard	100% of routes behind the key; CORS allowlisted
Data in git	89 files, 9.8 MB + 572 KB	0 files; .gitignore enforced
Module trees	3 (v1 root, src/, outreach/)	1; apps import packages, one direction
Pipeline stages wired	0 of 8 end to end	8 of 8, proof bundle mandatory
Pilot reply rate	no pilot	10% or better on 50 merchants
Human minutes per send	about 15	5 or less

Table 4: Go-live KPIs - the scoreboard for the four-week build.

- All five UI upgrades are merged with visual-parity screenshots.
- A WhatsApp note reaches a real merchant through the Cloud API with template, opt-out and proof bundle attached.
- A reply appears in Chatwoot and the lead stage updates in Twenty automatically.
- The pilot is running on 50 merchants with the three daily numbers being recorded.

7. Phase 5 - Day 30+: The Agent Trust Curve

Autonomy is earned, not configured - that is the masterclass's central lesson, condensed into its verification, trust, CI, auto-merge loop. IC turns that lesson into a five-stage ladder. A bot starts with zero autonomy and purchases the next stage with evidence: clean eval runs, zero unproven findings, zero rejected messages. The ladder matters because it replaces the two failure modes of amateur agent teams - permanent micromanagement, which wastes the automation, and premature autonomy, which burns real merchants with hallucinated claims - with a rule both humans and bots can point at.

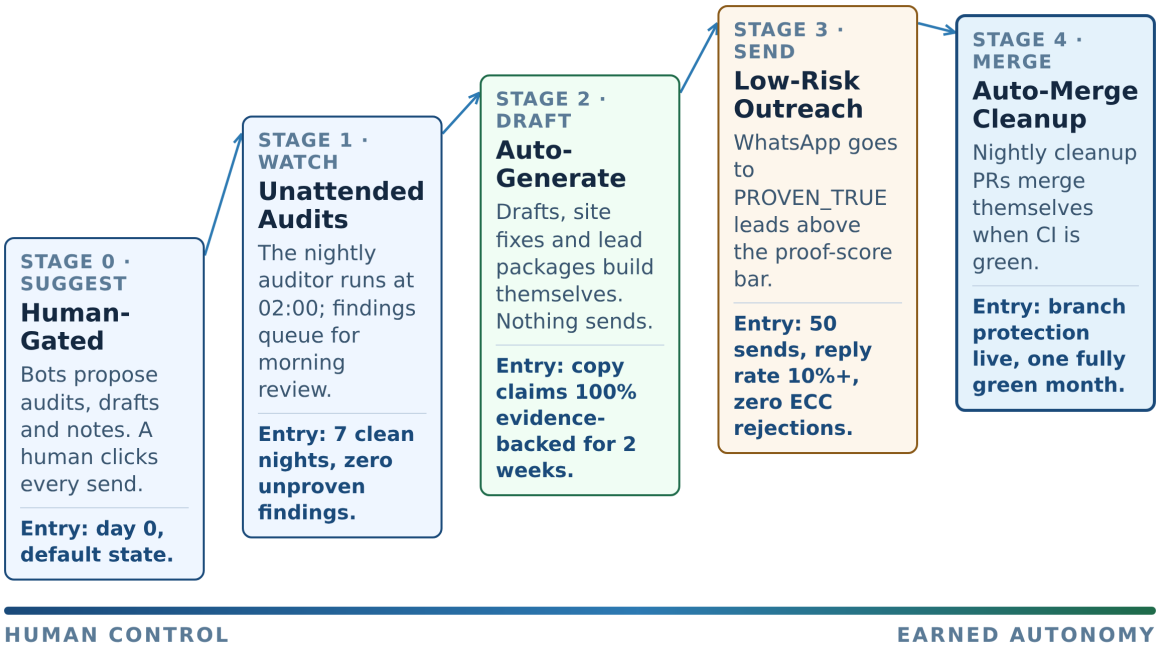


Figure 3: The five-stage trust curve - each stage is purchased with measured evidence, and any stage can be rolled back with one flag.

Stage	What the bot may do	Entry requirement
0 - Suggest	Propose audits, drafts and notes; a human clicks every send	Day 0, the default state
1 - Watch	The nightly auditor runs unattended; findings queue for morning review	7 consecutive nights with zero unproven findings
2 - Draft	Drafts, site fixes and lead packages build themselves; nothing sends	Copy claims 100% evidence-backed for 2 weeks
3 - Send	WhatsApp to PROVEN_TRUE leads above the proof-score bar, automatically	50 sends, reply rate 10% or better, zero ECC rejections for 2 weeks
4 - Merge	Nightly cleanup PRs merge themselves when the CI gate is green	Branch protection live, one fully green month

Table 5: The trust curve - autonomy is purchased with measured evidence, never with optimism.

Two rules keep the ladder honest. First, autonomy is per-scope, not per-bot: reaching stage 3 on clinics says nothing about a new vertical, which always starts back at stage 0 under the first-ten-leads rule of Chapter 9. Second, the rollback rule: any hallucination that reaches a customer, any ECC rejection spike, or any week where the CI gate goes red revokes the current stage immediately, and the bot re-earns it. The revocation is one flag in configuration, which is why the whole ladder rests on the walls of week 2.

> *A bot earns autonomy the way a junior earns push access: with a track record of verified work.*

8. The Operating Rhythm

Schedules are what turn agents into staff. The marketplace bots that inspired this architecture are not chat toys you summon; they hold a mandate and a cadence - the QA bot runs its acceptance checklists, the nightly auditor walks the codebase on a clock. IC already encodes this in the scheduled cadence engine modeled on the marketplace, and after go-live the rhythm below is the heartbeat: the machines run on schedule, and you spend thirty minutes a week steering instead of firefighting.

Bot	Mandate	Schedule	Output you read
ProjectsManagerBot	Queue boards, bottlenecks, workload balance	Continuous	Board health note on demand
NightlyAuditEngineer	Audit all stored leads: link health, TTFB drift, stale proofs; trigger Harness2 self-healing	Nightly, 02:00	Morning report at 07:00
QABot	Acceptance checklists on endpoints, API contracts, LLM compliance	Weekly, Monday	Pass/fail scorecard per service

Table 6: The standing bot roster from the scheduled cadence engine.

The weekly human ritual is thirty minutes, same day every week, and it has three agenda items. Review the week's PstackAuditBundle proof scores and read every bundle below the bar - a falling proof score is the earliest warning the system gives you. Adjust the trust curve: promote a bot that met its entry requirement, roll back one that hallucinated, and say out loud what evidence the promotion was based on. Approve the week's content changes: new pitch claims, new vertical profiles, new skills - the same approval gate you built for outbound messages, pointed at the system itself.

Monthly, zoom out. Rotate a slice of the eval set so the bots cannot overfit to yesterday's tests, prune skills nobody used, and re-run the twelve-dimension self-audit from Chapter 2 against the current codebase, scoring honestly and writing the numbers down. The scorecard you produce is the same figure that opened this plan, and watching it move is how you will know the machine is getting stronger rather than just busier.

9. The Vertical Expansion Recipe

Adding a vertical is deliberately a data change, not a code change - the architecture pays you back here for the discipline of the earlier weeks. Because detection, claims, routing and delivery are all configuration surfaces, a new category of merchant is a five-step recipe executed inside a single afternoon, followed by the observance period that earns its trust.

1. Add a detection profile to vertical_playbooks.py: the scrape signals that identify the pain (no booking widget, PDF menu, unanswered reviews, Instagram-only shop).
2. Add the vertical's pre-approved claims to pitch_claims.yaml - the CopywriterAgent may quote only these numbers, nothing generated.
3. Let detect_vertical() route new leads automatically; no agent code changes.
4. Let the deliverable router pick the offer: website, automation, or CRM setup.
5. Send the WhatsApp growth note through the same approval gate as every other vertical.

Priority	Vertical	Pain signal the audit detects	First offer
1	Clinics and dentists	No online booking; no-show pain; review backlog	Reminder automation + review funnel
2	Coaching and tuition	Enrollment by phone calls only	Admission funnel + batch reminders
3	Salons and spas	Bookings happen in DMs	Booking page + WhatsApp reminders
4	Restaurants and cafes	Menu is a PDF; ordering by phone	Mobile menu + ordering page
5	Real estate agents	Stale listings; no lead form	Listing landing + lead capture
6	Hotels and guesthouses	OTA commissions; no direct booking	Direct-booking page + concierge
7	Gyms and studios	Site exists, no trial form	Trial funnel + follow-up automation
8	Car dealers and detailers	Inventory lives in WhatsApp photos	Catalog page + enquiry bot
9	D2C and resellers	Instagram-only shop; broken checkout	Catalog cleanup + cart recovery

Table 7: The nine expansion verticals in recommended order, from the audit's market chapter.

The first-ten-leads rule protects reputation during expansion. For the first ten leads of any new vertical, the human reads the full audit and the exact WhatsApp note before anything sends, no matter how high the proof score is - detection models make category errors, and one message to the wrong business costs more than ten saved minutes. Only after ten clean sends does the vertical enter the trust curve at stage 0 like everyone else, and its detection precision is logged from that sample: at or above eighty percent correct classifications before any autonomy is discussed.

- Detection precision at or above 80% on a manually labeled sample of 30 businesses.
- Every pitch line traceable to pitch_claims.yaml and a PROVEN_TRUE bundle.
- Reply rate tracked per vertical from lead one, feeding the vertical's own trust-curve file.

10. The Library Map - Which File Matters When

Three documents, one job each: this playbook to do, the audit report for why, the playbook research for theory. Reading them in the wrong moment is how plans become reading clubs - so the map below says exactly when each one earns its open-tab cost, down to the chapter. The same logic applies inside the repository: the vendored skills are the team's training manuals, and six source files are the load-bearing walls.

Document / asset	Job	Open it when
This Master Action Plan	The ordered build order with acceptance checks	Every work session; print Chapter 11
Audit report - Ch. 2	Defects D1-D8 in full detail	While executing Phases 1-3
Audit report - Ch. 3	The 8 Slack principles of big-company engineering	When arguing process with a teammate
Audit report - Ch. 6	The 12 UI upgrades and 10 anti-slop laws	During week 4 and every UI PR after
Audit report - Ch. 8	Vertical deep-dives and market evidence	While running the expansion recipe
Agent Engineering Playbook	The Lauren Tan method, marketplace study, skill design	Designing a new bot or skill; onboarding
.agents/skills/pstack	Proof discipline: interrogate, blast-radius, visual-parity, hillclimb, show-me-your-work	Every code review and UI diff
.agents/skills/superpowers	TDD and debugging drills for agent-built code	Writing tests-first in every phase
.agents/skills/ecc	AgentShield scan rules + outbound hygiene	Week 2 CI setup; any outbound change
.agents/skills/harness2	Propose-probe-compose self-improvement loop	Monthly eval rotation (Chapter 8)

Table 8: The library map - do, why, theory, and the skill manuals in between.

Six source files are the load-bearing walls, and knowing them by name makes every phase faster. scheduled_cadence.py holds the bot roster and its clocks. evidence_verifier.py is the pstack principle at runtime. ecc_shield.py is the wall on both sides. vertical_playbooks.py and pitch_claims.yaml are where expansion lives as data. The HandoffEnvelope contracts module is the nervous system every agent speaks through. When something behaves strangely, look at these six before anywhere else; the audit found that almost every defect lived exactly where the architecture's discipline stopped.

11. The Master Checklist

Print this page. It is every acceptance check in the plan, in execution order, grouped by week; the checkbox column is deliberately ASCII so it survives any editor. A row is done when the evidence named in its phase exists, not when the code for it exists. When every row in the go-live block is checked, the four-week build is finished and the trust curve governs what happens next.

[ ]	Check	Week
[ ]	src/storage/repository.py written with LeadRepository and SessionRepository; 5 test files import cleanly	1
[ ]	pytest green with all files collected; README prints reproducible counts; no "51 passed" anywhere	1
[ ]	data/ and logs/ untracked and ignored; fresh clone carries no customer data	1
[ ]	scrapling and firecrawl pinned; missing engine fails loudly at boot	1
[ ]	Every API route behind the key; CORS allowlisted; reload off; /docs gated	2
[ ]	docker compose up runs api, worker and dashboard from a clean clone	2
[ ]	CI runs pytest, ruff and the ECC AgentShield scan; red build blocks merge	2
[ ]	LICENSE added, file:/// links replaced, v0.1.0 tagged	2
[ ]	One import tree remains; v1 and outreach/ trees deleted	3
[ ]	Dashboard split into ~15 modules; visual-parity screenshots match	3
[ ]	One scraper SDK with four flagged engines and red health checks	3
[ ]	No draft enters the Outbox without a PstackAuditBundle; proof score visible	3
[ ]	Five UI upgrades merged with parity screenshots; anti-slop PR template live	4
[ ]	WhatsApp Cloud API sends with template, opt-out and proof bundle	4
[ ]	Replies land in Chatwoot; lead stages update in Twenty	4
[ ]	Pilot running: 50 merchants, 2 verticals, 3 numbers recorded daily	4
[ ]	Trust-curve file exists per bot; rollback flag tested once on purpose	5
[ ]	Weekly 30-minute ritual on the calendar: proof scores, curve moves, approvals	5
[ ]	Monthly: eval slice rotated, skills pruned, 12-dimension self-audit re-scored	5
[ ]	First new vertical shipped via the 5-step recipe; first-ten-leads rule observed	5

Table 9: The Master Checklist - twenty checks from dishonest tests to earned autonomy.

The definition of done for the whole plan is one paragraph you can test with your own eyes: the test suite is green and the README matches it, the API rejects strangers, one command brings the product up, one import tree feeds one pipeline, every outbound message carries its proof, fifty real merchants have heard from you and their replies are in the inbox, and the bots run on schedule while you sleep - each one trusted exactly as far as its evidence, and no further. That is what industry-level means here: not a bigger demo, but a machine whose claims you can check and whose autonomy you can defend.